Learn Beyond

PROJECT REPORT

A report submitted in partial fulfillment of the requirements for the Award of Degree of

BACHELOR OF ENGINEERING

in

CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

StockFlow

Inventory and Stock Management System

"A single place to know what is in stock, what is moving and what needs replenishing."

Submitted by

SAI VARSHINI J (25AM104)

SAPARNA S (25AM105)

SASMITHA K M (25AM106)

SENAAN Y A (25AM107)

Under the supervision of

Mr. Ravi P

Assistant Professor, Department of CSE (AIML)

KPR Institute of Engineering and Technology, Coimbatore

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

AVINASHI ROAD, ARASUR, COIMBATORE – 641 407

2026

Learn Beyond

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the Project report titled **"StockFlow – Inventory and Stock Management System"** submitted by SAI VARSHINI J (25AM104), SAPARNA S (25AM105), SASMITHA K M (25AM106) and SENAAN Y A (25AM107) is work done by them and submitted during the academic year 2026 – 2027, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in CSE (Artificial Intelligence and Machine Learning).

Mentor	Skilling Officer	Head of the Department
Mr. Ravi P **Assistant Professor II,** **Department of CSE (AIML),** **KPR Institute of Engineering and Technology, Coimbatore.**	**Mrs. Sri Sakthi A** **Assistant Professor II,** **Department of CSE (AIML),** **KPR Institute of Engineering and Technology, Coimbatore.**	**Dr. Pandiya Rajan G** **Assistant Professor III,** **Department of CSE (AIML),** **KPR Institute of Engineering and Technology, Coimbatore.**

Place: Coimbatore

Date: October 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Dr. K. P. Ramasamy, Chairman**, for providing us with the opportunity and necessary facilities to undertake this project.

We extend our heartfelt thanks to **Dr. M. Akila, Principal**, for her valuable support, encouragement and guidance throughout the course of this project.

We would like to express our sincere thanks to **Dr. Pandiya Rajan G, Head of the Department, CSE (Artificial Intelligence and Machine Learning)**, for his valuable support, encouragement and permission to carry out this project.

We are deeply grateful to **Mrs. Sri Sakthi A, Skilling Officer**, for her valuable support, encouragement and guidance throughout the project.

We would like to express our sincere gratitude to our mentor, **Mr. Ravi P**, Department of CSE (Artificial Intelligence and Machine Learning), for his valuable guidance, continuous support and constructive suggestions throughout the course of this project.

We also extend our thanks to all the faculty members of the department, our friends and our families for their encouragement, cooperation and support. Their willingness to share knowledge and provide motivation contributed greatly to the successful completion of this project.

Finally, we express our heartfelt gratitude to everyone who directly or indirectly supported us in completing this project successfully.

ABSTRACT

Inventory is the working capital of every trading, retail and manufacturing organisation. Knowing how much of each item is available, where it is kept, what it cost and when it moved is essential for billing, purchasing, customer service and financial control. In small and medium-sized organisations these records are still kept in paper registers or in spreadsheets that are edited by several people. Such records become inconsistent, do not warn anyone when an item falls below its minimum level, and cannot answer simple questions such as how many units of a product were issued last week or which supplier is associated with an item.

This report presents StockFlow, a web-based inventory and stock management system that centralises the product catalogue, categories, suppliers and every stock movement in one application. The system provides a dashboard with live totals, low-stock alerts and quick actions; a searchable product catalogue with minimum and maximum stock levels; forms for receiving and issuing stock; a complete transaction history; operational reports; a low-stock queue with suggested reorder quantities; and an administrator-only user management screen. A React and TypeScript single-page interface communicates with a FastAPI REST API. Business rules, such as rejecting an issue that exceeds the available quantity, are enforced on the server, while SQLAlchemy stores users, products and movements in a SQLite database.

The application was verified through eight automated backend tests and a feature-by-feature walkthrough of the user interface. Product creation and validation, stock-in and stock-out flows, insufficient-stock rejection, low-stock and reorder calculations, dashboard statistics and role-based access were all confirmed. The report describes the domain, the limitations of existing methods, the system design (architecture, data model, stock algorithm and transaction policy), the implementation of each screen, and the testing results. It closes with the limitations of the current version and a roadmap for barcode support, multi-warehouse operation, notifications, forecasting and cloud deployment.

Keywords: inventory management, stock control, FastAPI, React, TypeScript, SQLAlchemy, SQLite, REST API, role-based access control, audit trail.

Chapter 1 Introduction

1.1 Background

Every organisation that buys, stores and sells goods depends on accurate stock information. A retail shop needs to know whether a popular item is available before promising it to a customer. A warehouse needs to know how many units were received from a supplier and how many were issued to a department. An accounts team needs the value of the inventory that is held. When these answers are kept in memory or in a notebook, they are rarely consistent, and the cost of the resulting mistakes is paid in lost sales, emergency purchases and duplicated orders.

The two classic inventory failures pull in opposite directions. Overstocking ties up capital, fills valuable storage space and increases the risk of damage or obsolescence. Understocking interrupts sales and production, forces expensive rush orders and damages the reputation of the business. The balance between the two is managed through minimum and maximum stock levels: the minimum signals when replenishment must begin, and the maximum caps how much should be held. A useful system must therefore track quantities continuously and make exceptions visible before they become shortages.

Spreadsheets are the most common digital alternative, and they are flexible, but they were not designed for transactional data. Several people editing one file create conflicting versions, formulas can be overwritten accidentally, and there is no reliable record of who changed a quantity or why. Desktop inventory packages solve part of the problem but often require a licence, a local installation and a training period. A browser-based application with a clear data model, server-side validation and an audit trail offers a practical middle ground for small teams and for academic demonstrations of a complete inventory workflow.

StockFlow was built on this idea. It treats every receipt and issue as a transaction, keeps the current balance on the product record, derives the stock status from the minimum level, and stores the identity of the user who performed each movement. The interface is deliberately compact: a dashboard for the daily picture, a catalogue for maintenance, two focused forms for movement, an audit table and four reports. The result is a system that can be demonstrated and extended without hiding its logic behind a commercial platform.

1.2 Motivation

The motivation for the project came from the practical difficulties observed in small warehouses and college stores:

- Poor visibility: staff cannot answer basic questions such as how many units remain, which items are below their minimum level, or how much inventory is held, without manually counting or searching several files.
- Weak audit trail: when a quantity changes, there is no reliable record of the previous balance, the new balance, the reference document or the person responsible.
- Manual calculations: stock-in and stock-out arithmetic, inventory valuation and reorder quantities are repeated by hand, which introduces avoidable errors.
- Fragmented information: product details, supplier contacts and movement records live in separate documents, so cross-checking them is slow.
- No role separation: in shared spreadsheets every user can change every value, including prices and user lists, with no restriction.
- Reporting effort: producing a low-stock list or a supplier summary requires filtering and reformatting, which is usually postponed until it is needed urgently.

A second motivation was educational. The project offered an opportunity to connect a modern frontend, a typed backend and a relational database into one working system, and to apply software engineering ideas such as layered design, validation, authentication, automated testing and responsive interface design to a problem that is easy to explain and easy to evaluate.

1.3 Objectives

The objectives of StockFlow are to:

- provide a single, browser-based workspace for products, categories, suppliers and stock movements;
- maintain an accurate current balance for every product and derive its In Stock, Low Stock or Out of Stock status automatically;
- record every receipt and issue as an immutable transaction containing the previous balance, the new balance, the quantity, the user, a reference and the time;
- reject invalid operations such as a non-positive quantity, a duplicate product code, an issue larger than the available stock, or a maximum level that is not greater than the minimum level;
- present a dashboard with inventory totals, low-stock alerts and reorder suggestions so that replenishment can be planned;

- generate inventory, stock-movement, low-stock and supplier reports directly from the live database;
- separate administrative operations from day-to-day stock operations through role-based access control; and
- offer a responsive interface that works on a computer, tablet or mobile browser without installation on the client.

1.4 Scope and assumptions

The scope of this version is a single-organisation inventory system that runs on a local network or on one machine for demonstration. The following scope and assumptions apply:

- The system manages products, categories, suppliers, users, stock-in movements, stock-out movements and reports. It does not manage sales orders, invoices, payments, purchase orders or accounting entries.
- Stock is tracked as a single quantity per product. Batch numbers, serial numbers, expiry dates, bin locations and multiple warehouses are outside the current scope.
- Authentication is implemented with JSON Web Tokens and bcrypt password hashes. In the delivered demonstration build the interface starts directly as the demo administrator; the login API remains available for API clients.
- The backend and the SQLite database run on the same machine. The database file is created automatically on first start and can be populated with the supplied seed script.
- It is assumed that users enter correct product, supplier and movement information and that the server clock is correct, because transaction timestamps and report periods depend on it.
- The system is a record-keeping and decision-support tool. It does not provide financial advice, tax calculations, medical advice or any other domain-specific recommendation.
- Out of scope for this version: barcode scanning, e-mail or SMS notifications, multi-warehouse transfers, demand forecasting, supplier self-service, document attachments and native mobile applications. These are proposed as future work in Chapter 7.

Chapter 2 Domain Review

2.1 Existing Systems

Organisations currently keep track of stock in one of the following ways:

Manual registers. A notebook or a set of bin cards records receipts and issues. The method is inexpensive and works without electricity, but it is slow, difficult to total, vulnerable to lost pages and almost impossible to analyse across products or periods.

Spreadsheets. They are the most widespread digital tool: they support formulas, filters and simple charts, and every office already has one. Their weaknesses appear as soon as more than one person edits the file: there is no transaction log, no server-side validation, no concurrency control and no reliable link between a movement and the user who recorded it. A single overwritten formula can silently corrupt the balance.

ERP and commercial inventory suites. Enterprise resource planning and inventory suites provide complete functionality for large organisations, including purchasing, warehousing, accounting and manufacturing. They are powerful but expensive, require configuration and training, and impose far more process than a small store or a college department needs.

Billing and point-of-sale applications. Many billing applications include a stock module. These tools are convenient for a shop that already uses them for invoicing, but the inventory features are usually secondary: minimum levels, reorder suggestions, movement references, role separation and supplier reports may be missing or locked behind a higher plan.

Custom in-house tools. Some teams build a small in-house database or script. This can fit the organisation precisely, but it creates a maintenance obligation: schema changes, backups, security patches and user support must be handled internally, and the original author may not be available later.

The common thread is that each option solves part of the problem. Registers describe but do not calculate, spreadsheets calculate but do not control, ERP suites control but at a cost and complexity that small teams cannot justify.

2.2 Stock Movement and Dynamic Inventory

Inventory is not a static number. It changes whenever goods are received, issued, returned, damaged or counted. A system that stores only a quantity per product cannot explain how that quantity was reached, so corrections overwrite history and discrepancies cannot be

investigated. StockFlow therefore treats the product record as the current balance and the transaction table as the ledger. Every operation writes both: the balance is updated and the movement is appended with the previous and new quantities. The two values can be reconciled at any time.

Dynamic behaviour is required in everyday situations. A supplier delivers fewer units than ordered, a department returns unused items, a physical count reveals damage, or a product price changes. Table 2.1 summarises how the system responds to the events that occur most often.

Table 2.1: Typical inventory events and the response of the system

Situation	Response of the system
Goods received from a supplier	The quantity is added to the current balance, the purchase price can be updated, and a STOCK_IN transaction is recorded with the reference and user.
Items issued to a department or customer	The requested quantity is checked against the available stock, the balance is reduced, and a STOCK_OUT transaction is recorded with the department in the notes.
Damaged or expired units	The units are issued with an explanatory note, so the loss is visible in the audit trail instead of disappearing in an adjustment.
Physical count correction	The difference is entered as a stock-in or stock-out movement with a reference such as COUNT-2026-04, preserving the reason.
Purchase price change	The product can be edited directly, or the new price can be supplied during a stock-in operation and stored on the product and the transaction.
Product discontinued	The product is removed through the admin-only delete action; its dependent transactions are deleted by the configured cascade and the remaining catalogue is unaffected.

2.3 Proposed System

StockFlow is a web-based inventory and stock management system with a React single-page interface and a FastAPI backend. The user works in one browser tab: the sidebar switches between the dashboard, catalogue, movement forms, audit trail, reports and administration. The interface calls a versioned REST API, the API validates every request with Pydantic schemas and business rules, and SQLAlchemy persists the result in SQLite. Because the browser holds no business data, two users who open the same address see the same records after a refresh.

The main features of the proposed system are:

- a dashboard with six metric cards (products, units in stock, inventory value, suppliers, low stock and out of stock), a low-stock alert panel and quick actions;

- a searchable product catalogue with category and supplier names, minimum and maximum levels, colour-coded status badges, and edit and delete actions for administrators;
- maintenance screens for categories and suppliers, including contact, address and GST details;
- dedicated Stock In and Stock Out forms that validate quantities, capture references and notes, and update the balance atomically;
- a transaction ledger that records the previous balance, the new balance, the user, the type, the quantity and the reference for every movement;
- four reports (inventory, stock movement, low stock and suppliers) generated from the live database, plus a low-stock queue with suggested reorder quantities;
- administrator-only user management with roles, active status and password changes; and
- a responsive layout that collapses the sidebar, reflows cards and allows tables to scroll on small screens.

2.4 Summary of Existing Approaches

Tables 2.2 and 2.3 compare the approaches described above with the proposed system. The comparison is qualitative: it considers the features that matter for a small inventory operation rather than the total feature count of a commercial ERP product.

Table 2.2: Strengths and limitations of existing approaches

Approach	Strength	Limitation
Manual register	No cost, simple to start	Slow, no validation, no reports, easily lost
Spreadsheet	Familiar and flexible	No audit trail, no concurrency, formula errors
ERP suite	Complete and integrated	Expensive, complex, heavy training and setup
Billing application	Convenient for sales	Inventory features are secondary or limited
Custom in-house tool	Fits the organisation	Maintenance and security burden
StockFlow (proposed)	Catalogue, movements, alerts and reports in one place	Single-organisation, local deployment in this version

Table 2.3: Feature comparison of existing approaches with the proposed system

Feature	Manual register	Spreadsheet	ERP suite	Billing app	StockFlow
Stores product details	Yes	Yes	Yes	Partial	Yes
Validates every movement	No	Partial	Yes	Partial	Yes
Minimum / maximum levels	Manual	Manual	Yes	Rare	Yes
Audit trail with user	No	No	Yes	Partial	Yes
Role-based access	No	No	Yes	Partial	Yes
Low-stock and reorder list	Manual	Manual	Yes	Rare	Yes
Operational reports	No	Manual	Yes	Yes	Yes
Setup cost and effort	Very low	Low	High	Medium	Low

Chapter 3 Gap Identification and Problem Statement

3.1 The Gap

The review shows that each existing option solves only part of the inventory problem. Registers and spreadsheets record data but do not enforce rules, warn about shortages or preserve a trustworthy history. Commercial suites enforce rules but are disproportionate for a small store, a laboratory or a college department. The specific gaps are:

- Limited visibility: there is no single view of the current quantity, its value and its status relative to the minimum level.
- No reliable audit trail: a changed quantity cannot be traced to a movement, a reference document and a user.
- No proactive alerts: low-stock situations are discovered during a count or after a sale has already failed, not when the threshold is crossed.
- Error-prone arithmetic: balances and inventory values are calculated manually, and issues larger than the available stock are not always prevented.
- Weak access control: shared files allow any user to change prices, delete products or edit the user list.
- Fragmented reporting: inventory, movement and supplier information must be assembled from separate files, so routine reporting is delayed.
- High cost of adequate tools: the systems that do provide these features are priced and designed for larger organisations.

3.2 Problem Statement

Small organisations need to know, at any moment, what stock is available, what it is worth, what moved and what must be reordered. Manual registers and spreadsheets cannot enforce stock rules or provide a reliable audit trail, while commercial inventory suites are too expensive and too complex for the scale of the operation. Hence, a lightweight inventory and stock management system is required that records products, suppliers and movements in a relational database, validates every operation on the server, derives stock status and reorder quantities automatically, presents the information through a clear dashboard and reports, and restricts administrative actions to authorised users.

3.3 Contribution

The contributions of this work are:

- a complete, browser-based inventory workspace that combines catalogue maintenance, stock movement, audit history, reports and user administration;
- a transaction-centred stock model in which every receipt and issue stores the previous balance, the new balance, the quantity, the reference and the responsible user;
- server-side validation of product codes, stock levels, quantities and relationships, so that invalid data cannot enter the database through the interface;
- automatic derivation of stock status and reorder quantities from minimum and maximum levels, exposed through dashboard alerts, a low-stock queue and reports;
- role-based access control with hashed passwords and signed tokens, plus a self-protection rule that prevents an administrator from deleting, deactivating or demoting their own account; and
- a responsive React interface and a documented REST API that can be extended with barcode input, notifications or a different database without rewriting the domain logic.

Taken together, these contributions address the gaps identified in Section 3.1: the system makes stock visible, preserves the history of every change, warns before a shortage becomes critical and limits destructive actions to administrators. The next chapter turns these ideas into a concrete architecture, data model and set of algorithms.

Chapter 4 System Design

4.1 Architecture

StockFlow uses a three-tier architecture. The presentation tier is a React single-page application that runs in the browser; the application tier is a FastAPI service that owns the business rules; and the data tier is a SQLite database accessed through SQLAlchemy. Each tier has one responsibility, and the boundaries are explicit: the browser never sends SQL, and the server never renders HTML. This separation makes the system easier to test, because the stock rules can be exercised through the API without a browser, and easier to replace, because the database can be changed without rewriting the interface.

Table 4.1: Layers of the implemented application

Layer	Responsibility
Presentation layer (React, TypeScript, CSS)	Dashboard, catalogue, movement forms, audit tables, reports and administration screens; search, modal forms, status badges and responsive layout. Communicates only through the REST API.
Application layer (FastAPI, Pydantic, services)	Routing, authentication and role checks, request validation, stock-in and stock-out rules, dashboard aggregation and report generation.
Data layer (SQLAlchemy, SQLite)	Persistence of users, categories, suppliers, products and transactions; foreign keys, relationships, cascades and transactions.
Security layer (JWT, bcrypt, dependencies)	Password hashing, token creation and verification, active-user checks, admin-only dependencies and the demo-session fallback used by the delivered build.

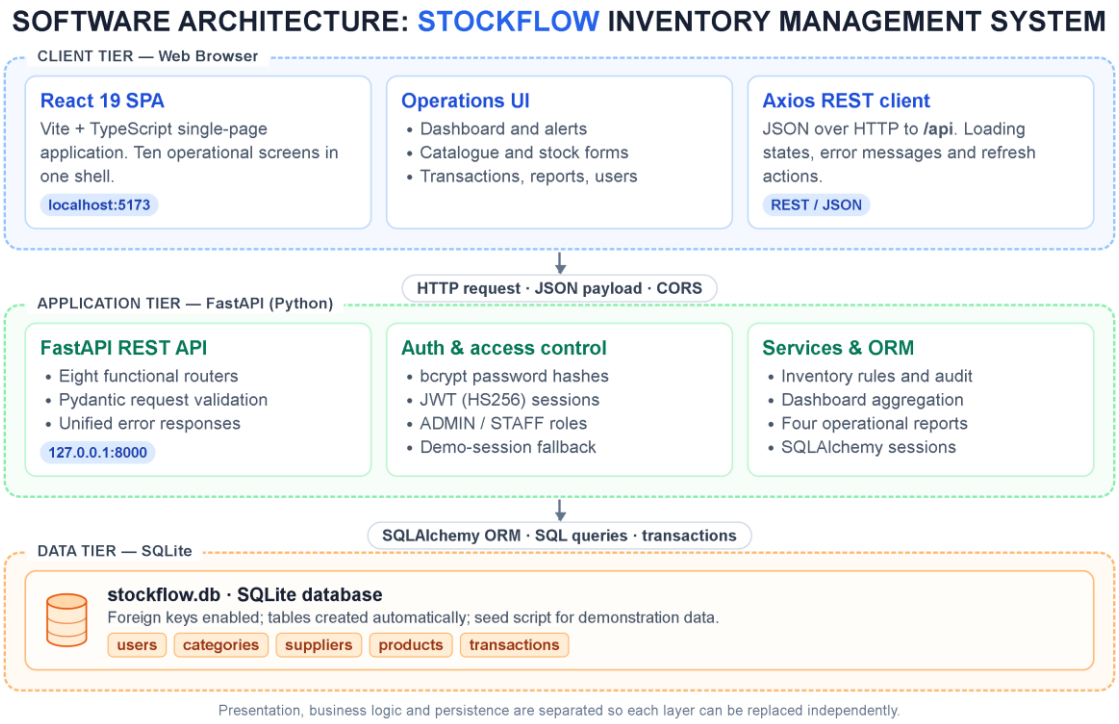


Fig 4.1: Three-tier software architecture of StockFlow

The client tier is served by Vite during development at http://localhost:5173. Axios sends JSON requests to the API at http://127.0.0.1:8000/api. Every response is either the requested resource or a JSON object with a human-readable detail message, which the interface displays in an alert instead of failing silently. A small set of shared functions keeps the screens consistent: Indian rupee formatting, date formatting, stock-status labels and a request-error extractor.

The application tier is organised by router: authentication, users, categories, suppliers, products, inventory, dashboard and reports. Routers depend on shared services for stock movements, dashboard statistics, report generation and database sessions. Pydantic schemas validate the shape of every request body and query parameter before the handler runs, and FastAPI converts validation failures into a single readable message. The security dependency resolves the current user, checks that the account is active, and exposes an admin-only variant for destructive operations.

The data tier is a single SQLite file, stockflow.db, with foreign keys enabled on every connection. SQLAlchemy models define the columns and relationships, and sessions are created per request and closed in a finally block. Because SQLite is embedded, the demonstration needs no database server; because the access layer is SQLAlchemy, the same models can be moved to PostgreSQL for a multi-user deployment with little change to the service code.

4.2 Data Model

The data model centres on five entities. A category groups products, a supplier provides them, a user performs movements, and a transaction records each movement of a product. Table 4.2 lists the attributes that matter for the design; timestamps are present on every table and are omitted from the descriptions for brevity.

Table 4.2: Entities and attributes of the data model

Entity	Key attributes	Description
USERS	id, name, email, password_hash, role, is_active	Account that signs in, performs movements and, for administrators, manages other users.
CATEGORIES	id, name, description	Logical group used for catalogue navigation and inventory reporting.
SUPPLIERS	id, name, contact_person, phone, email, address, city, state, gst_number	Vendor that supplies products; multiple products may share one supplier.

Entity	Key attributes	Description
PRODUCTS	id, product_code, name, description, category_id, supplier_id, purchase_price, selling_price, current_stock, minimum_stock, maximum_stock, location	Master record of an item, including its current balance and replenishment thresholds.
TRANSACTIONS	id, product_id, user_id, transaction_type, quantity, previous_stock, new_stock, unit_price, reference_number, notes, created_at	Immutable ledger entry for every receipt or issue, used by the audit trail and reports.

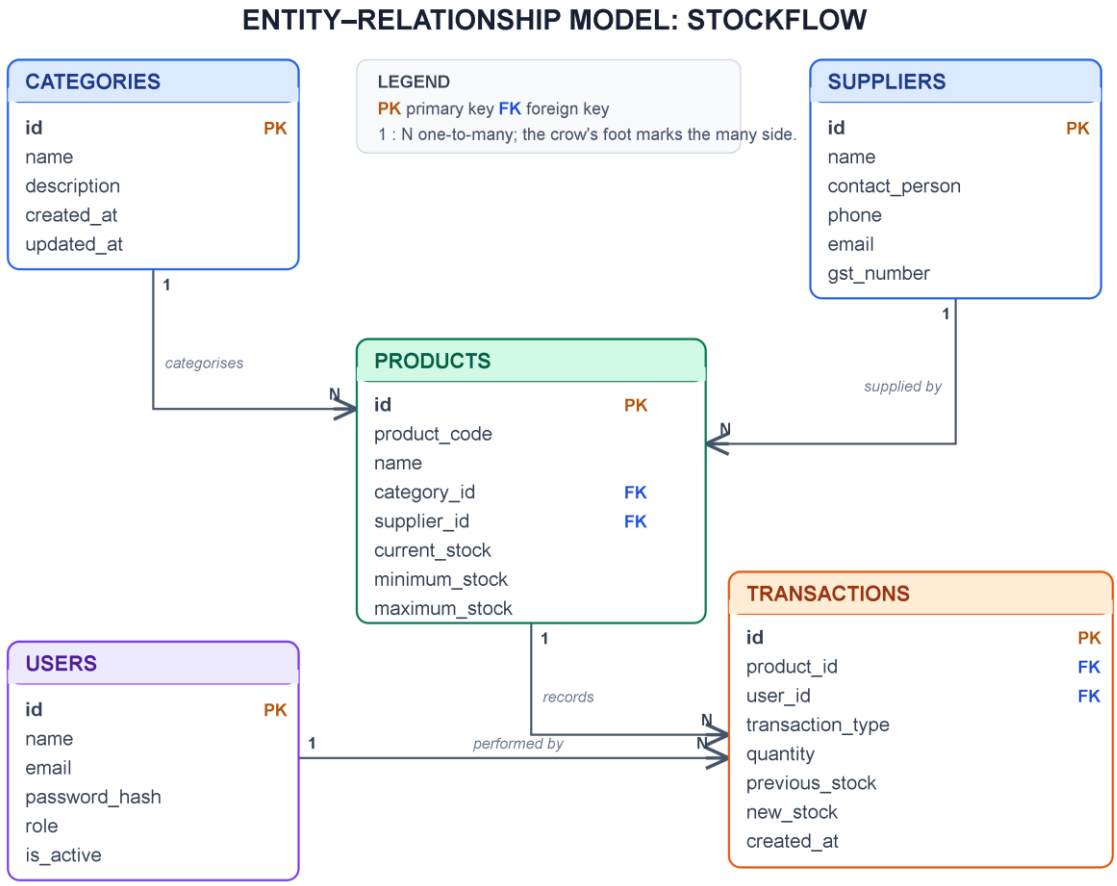


Fig 4.2: Entity-relationship model of the StockFlow database

The relationships enforce the operational rules of the domain. One category can contain many products, and one supplier can provide many products; both foreign keys use ON DELETE RESTRICT, so a category or supplier that is still referenced cannot be removed by accident. One product can have many transactions, and the relationship is configured with a cascade so that deleting a product also removes its movement history; this keeps the ledger consistent with the catalogue but means that deletion is a destructive, admin-only action. One user can perform many transactions, and the foreign key uses ON DELETE SET NULL so that historical movements survive if an account is removed. Stock status is not stored as a column: it is derived from current_stock and minimum_stock whenever a product is read, which prevents the stored status and the actual balance from disagreeing.

4.3 Inventory Movement Algorithm

The movement algorithm is the operational core of the system. It receives a product, a quantity, optional price and reference information, validates the request, updates the balance and appends a transaction in one database commit. The steps are:

- Receive the product identifier, the quantity and the optional price, reference and notes.
- Load the product; if it does not exist, return a 404 error with a readable message.
- For a stock-in movement, reject a quantity that is not greater than zero.
- For a stock-out movement, reject a quantity that is not greater than zero and reject a quantity larger than the available balance.
- Store the previous balance, calculate the new balance by addition or subtraction, and write it to the product.
- For stock-in, update the purchase price when a non-negative unit price is supplied.
- Create a transaction with the type, quantity, previous balance, new balance, unit price, reference, notes and the current user.
- Commit the product update and the transaction together, so a failure leaves neither change behind.
- Refresh the product and return the transaction to the caller.
- Let the dashboard, low-stock queue, reports and product status derive their values from the new balance on the next read.

Flow: select product and operation → validate quantity → read previous balance → add or subtract → write transaction → commit → refresh dashboard and reports

Algorithm 4.1: Stock movement and status derivation (simplified)

```
FUNCTION stock_in(product_id, quantity, unit_price):
    product <- FIND Product WHERE id = product_id
    IF product IS NULL THEN RETURN 404
    IF quantity <= 0 THEN RETURN 400
    previous <- product.current_stock
    product.current_stock <- previous + quantity
    IF unit_price IS PROVIDED THEN product.purchase_price <- unit_price
    INSERT Transaction(STOCK_IN, quantity, previous, new_stock, user)
    COMMIT; RETURN transaction

FUNCTION stock_out(product_id, quantity, reference):
    product <- FIND Product WHERE id = product_id
    IF product IS NULL THEN RETURN 404
    IF quantity <= 0 OR quantity > product.current_stock THEN RETURN 400
    previous <- product.current_stock
    product.current_stock <- previous - quantity
    INSERT Transaction(STOCK_OUT, quantity, previous, new_stock, user)
    COMMIT; RETURN transaction

FUNCTION status(product):
    IF current_stock = 0 THEN RETURN "OUT_OF_STOCK"
    IF current_stock <= minimum_stock THEN RETURN "LOW_STOCK"
    RETURN "IN_STOCK"
```

Several rules are enforced before the transaction is written. Product codes are unique and are stored in upper case, so PRD-KBD-03 and prd-kbd-03 cannot coexist. Prices and quantities are validated as non-negative, and a product cannot be created or updated with a maximum level that is not strictly greater than its minimum level. Category and supplier identifiers must refer to existing rows. The stock-out check compares the requested quantity with the current balance inside the same request, so an issue that would drive the balance below zero is rejected with the available quantity in the message. These checks are implemented on the server, not only in the browser, so they also protect the API when it is called by another client.

The initial stock of a new product is handled specially: if a positive initial quantity is supplied, the product is created and an opening STOCK_IN transaction is written with the reference INITIAL_OPENING_STOCK. This keeps the audit trail complete from the first unit and avoids a product whose balance is not explained by any movement.

4.4 Transaction and Event Policy

Inventory changes are event-driven. Every event updates the relevant entity, writes or reuses an audit entry, and relies on derived values rather than duplicated state. Table 4.3 lists the principal events and their effect.

Table 4.3: Events, state changes and validation

Event	Effect on state	Validation / follow-up
Stock received	Balance increases; STOCK_IN transaction appended	Quantity > 0; price non-negative; product must exist
Stock issued	Balance decreases; STOCK_OUT transaction appended	Quantity > 0 and <= available balance
Product created with opening stock	Product inserted; opening STOCK_IN transaction appended	Unique code; valid category and supplier; max > min
Product edited	Product fields updated; next reads derive a new status	Unique code if changed; max > min; valid relationships
Product deleted	Product removed; dependent transactions removed by cascade	Admin only; confirmation in the interface
Category or supplier edited	Related names and report rows reflect the change	Name uniqueness for categories; valid fields
Category or supplier deleted	Removed only when no product references it	ON DELETE RESTRICT returns an error otherwise
User deactivated	Account cannot authenticate; historical transactions remain	Admin only; cannot deactivate own account
Page opened or refreshed	Dashboard, catalogue, alerts and reports are read from the database	Demo session or bearer token resolves the current user

The policy has two important consequences. First, a movement and the balance it produces are committed together, so a crash between the two writes cannot leave the ledger and the product record out of step. Second, status and reorder quantities are recomputed from the current balance and the thresholds instead of being stored. This avoids the classic inconsistency in which a product is marked In Stock while its quantity is zero, and it means that changing the minimum level immediately changes the alerts and the reports.

4.5 Performance Indicators

The system is judged both as an inventory control tool and as a software product. The inventory indicators in Table 4.4 can be computed from the product and transaction data, and they give the organisation a way to evaluate whether the system is improving stock discipline.

Table 4.4: Performance indicators

Indicator	Formula	Interpretation
Stock accuracy	Products whose recorded balance equals the physical count / total products x 100	Higher is better; shows whether the ledger can be trusted
Stockout rate	Products with zero stock / total products x 100	Lower is better; high values indicate weak replenishment
Low-stock ratio	Products at or below minimum / total products x 100	Lower is better; reviewed through the low-stock queue
Inventory value	Sum of current_stock x purchase_price over all products	Working capital held in stock; shown on the dashboard
Low-stock response time	Time between crossing the minimum level and the next stock-in	Lower is better; measures replenishment discipline
Audit completeness	Transactions with previous balance, new balance and user / total transactions x 100	Should be 100% in this design

Software performance is measured separately: the latency of the API endpoints, the time required to build the frontend, the outcome of the automated test suite and the behaviour of the interface when the backend is unavailable. Chapter 6 reports the measured values and the limitations of the evaluation.

Chapter 5 Implementation

5.1 Technologies Used

The implementation uses a small, well-documented set of technologies. The frontend is a TypeScript single-page application built with Vite, and the backend is a Python service built with FastAPI. SQLAlchemy maps the object model to SQLite. This combination keeps the development loop fast while still exercising the concerns of a real multi-tier application: HTTP, JSON, validation, authentication, relational constraints and responsive interface design.

Table 5.1: Technology stack

Technology	Use in the project
React 19 + TypeScript	Component-based interface, typed props and state for products, categories, suppliers, users and transactions.
Vite 8	Development server with hot module replacement, production bundling and a proxy for /api requests.
CSS and Tailwind pipeline	Custom stylesheet for the application shell, cards, tables, forms, modals and media queries; Tailwind directives remain available for utility styling.
Axios	HTTP client for the REST API with centralised error extraction and JSON handling.
Lucide React	Consistent outline icons for navigation and actions.
FastAPI	REST routers, dependency injection, automatic OpenAPI documentation and exception handling.
Pydantic 2	Request and response schemas with field constraints and cross-field validation.
SQLAlchemy 2.0	ORM models, sessions, relationships, cascades and transactional writes.
SQLite	Embedded relational database stored in backend/stockflow.db; foreign keys are enabled per connection.
PyJWT + bcrypt	Signed access tokens and adaptive password hashing.
Uvicorn	ASGI server used to run the API on port 8000.
pytest + HTTPX	Automated API tests against an in-memory SQLite database.

The interface follows a consistent visual language: a dark navy sidebar, a white content surface, a blue primary action colour, rounded cards, compact tables and colour-coded status badges. Green badges mark In Stock, amber badges mark Low Stock and red badges mark Out of Stock. Every screen uses the same toolbar, spacing and typography, so a user who has learned the product form can predict how the supplier form behaves. The typography is deliberately small and dense, because inventory work involves comparing many values at once.

5.2 Application Shell and Dashboard

After the frontend loads, it requests /api/auth/me to resolve the current user. In the delivered build the API answers with the demo administrator, so the application opens directly on the dashboard. The shell is a fixed sidebar and a sticky top bar. The sidebar contains the ten operational screens, the signed-in user and the role; the top bar contains the menu toggle, the page title, a refresh button and the user avatar. The menu toggle collapses the sidebar to icons, which is useful on narrow screens and when a wide table needs more room.

The dashboard is the landing screen and the daily summary. A gradient hero panel provides a short greeting and a Record stock action. Six metric cards follow: total products, units in stock, inventory value at purchase price, suppliers, products at or below their minimum level, and products with zero stock. The values come from a single dashboard endpoint that aggregates the product table, so the cards cannot disagree with each other.

Below the cards, the low-stock alerts panel lists the products that need attention, ordered by current quantity, with the product code, minimum level and units remaining. The View all action opens the full low-stock queue. The quick actions panel provides one-click routes to Stock In, Stock Out, Products and Transactions. Figure 5.1 shows the dashboard with the seeded demonstration data: 14 products, 434 units, an inventory value of Rs. 34,119, five suppliers, four low-stock items and one out-of-stock item.

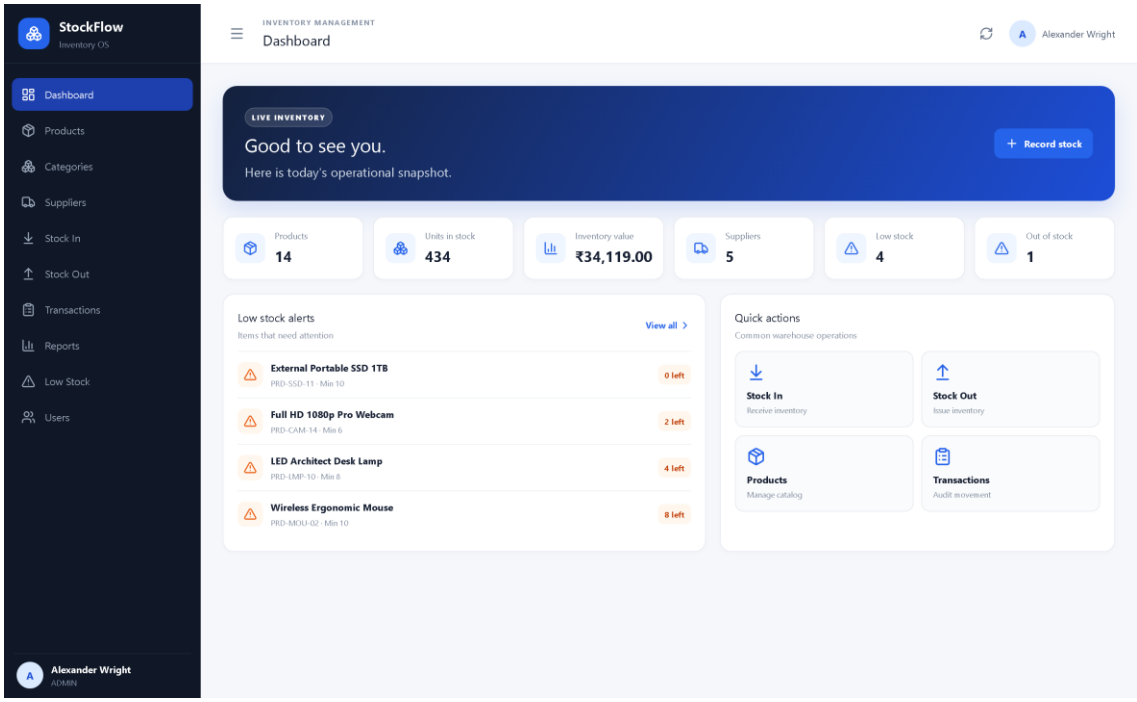


Fig 5.1: Dashboard with summary cards, low-stock alerts and quick actions

5.3 Catalogue Management

5.3.1 Products List

The Products page is the maintainable view of the catalogue. The table shows the product name and code, category, supplier, current quantity with the minimum and maximum levels, selling price, a status badge and, for administrators, edit and delete buttons. The search box filters by product name or code on the server after a short debounce, so typing does not flood the API. The page header reports how many products are shown and the list is ordered by creation time, newest first, which keeps recently added items visible.

Status is presented as a badge rather than a raw database value: IN_STOCK becomes a green In Stock label, LOW_STOCK an amber Low Stock label and OUT_OF_STOCK a red Out of Stock label. This is the screen where a storekeeper starts the day, because it combines product identity, availability and price in one row. Figure 5.2 shows the catalogue with its search field and action column.

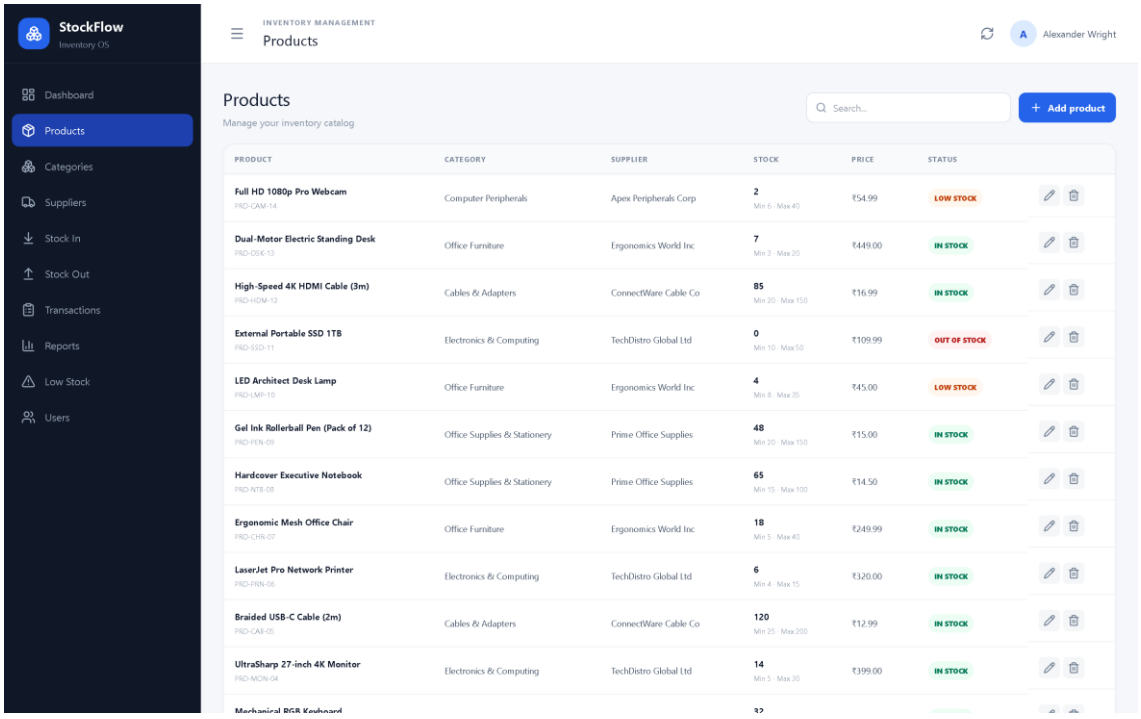


Fig 5.2: Product catalogue with search, stock levels, status badges and row actions

5.3.2 Add and Edit Product

The product form is presented in a modal over the catalogue, so the user does not lose the filter or scroll position. It uses a two-column grid for short fields and a full-width description area. Required fields are enforced by the browser and re-checked by the API. When editing, the form is pre-filled with the stored values; the initial stock field is shown

only for new products, because later quantity changes must go through the movement forms and produce an audit entry.

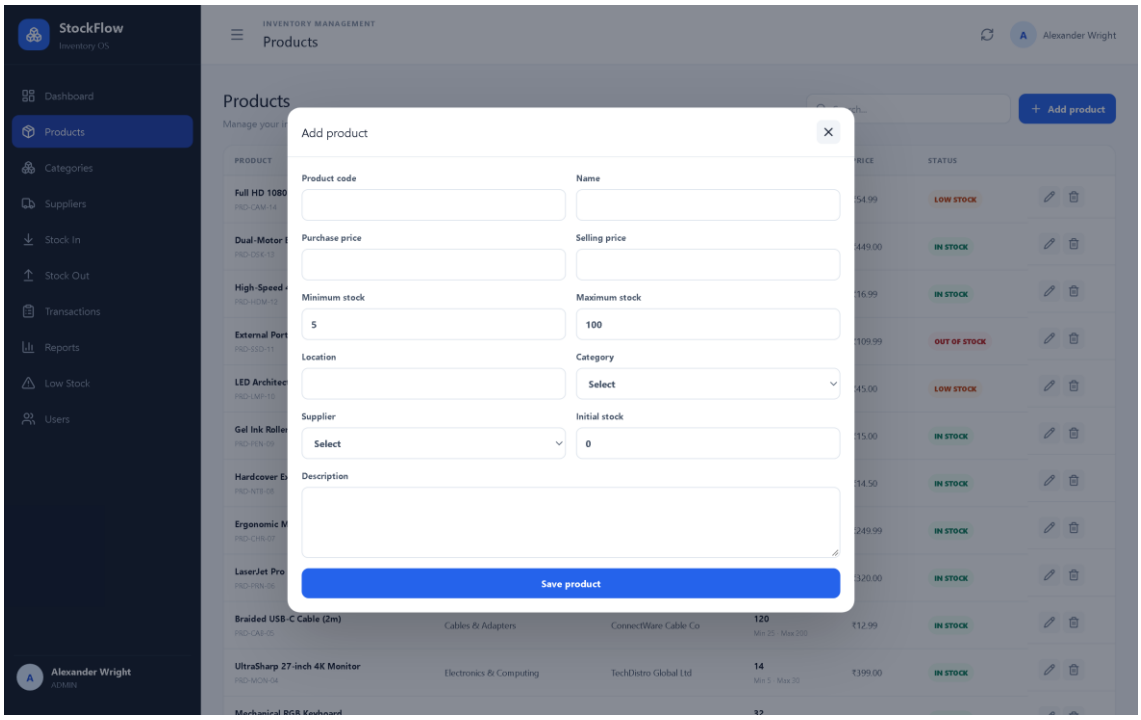


Fig 5.3: Add product modal with catalogue and supplier selection

Table 5.2: Fields of the product form

Field	Required	Purpose and validation
Product code	Yes	Unique business identifier, 2-50 characters, stored in upper case.
Name	Yes	Descriptive product name, 2-200 characters.
Category	Yes	Must reference an existing category.
Supplier	Yes	Must reference an existing supplier.
Purchase price	Yes	Cost per unit, zero or positive; used for inventory valuation.
Selling price	Yes	Sale price per unit, zero or positive; shown in the catalogue.
Initial stock	No (create only)	Zero or positive; a positive value creates an opening STOCK_IN transaction.
Minimum stock	Yes	Threshold at or below which the product is Low Stock; default 5.
Maximum stock	Yes	Replenishment ceiling and basis for the suggested reorder; must exceed the minimum.
Location	No	Free-text storage location, up to 100 characters.
Description	No	Free-text notes about the item.

Saving the form calls POST /api/products or PUT /api/products/{id}. The API rejects a duplicate code, a missing category or supplier, a negative price, a negative quantity and a maximum level that is not greater than the minimum. A successful create returns the product with its derived status and inventory value, and the interface reloads the catalogue

and shows a confirmation toast. Editing works the same way, except that the stock balance is not accepted from the form.

5.3.3 Categories and Suppliers

Categories are managed as a set of cards. Each card shows the category name, its description, the number of products that use it and, for administrators, an edit action. The product count is computed with an outer join, so a category with no products still appears with a count of zero. The API prevents a duplicate name and refuses to delete a category that still contains products.

Suppliers are managed in a table with the supplier name, city and state, contact person, phone, e-mail, product count and an edit action. The form stores the address and GST number in addition to the contact details. These fields matter for purchasing, because the same supplier may be contacted about a delivery, a price change or a tax document. Supplier deletion is similarly blocked while products reference the supplier.

5.4 Stock Operations

Stock In and Stock Out are separate screens because they answer different operational questions. The Stock In form receives inventory: the user selects a product, enters the quantity, optionally records the unit purchase price, a reference number and notes, and confirms. If a purchase price is supplied it is written to the product, so the next inventory valuation reflects the latest cost. The product selector shows the current quantity for each item, which helps the user confirm that the correct product was chosen.

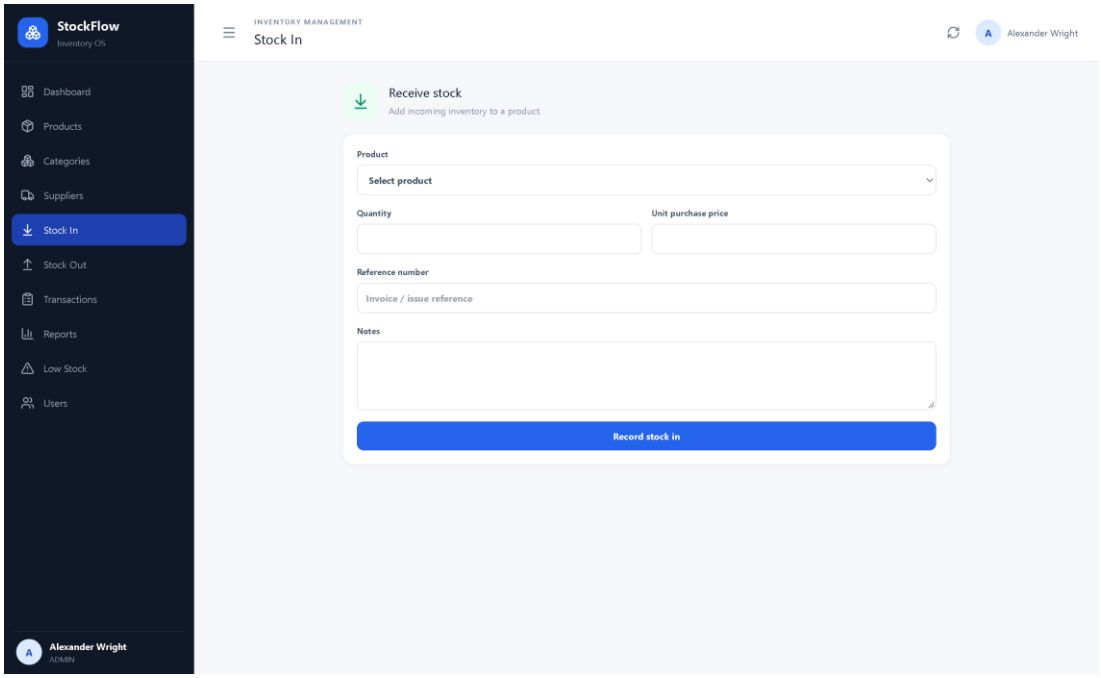


Fig 5.4: Stock In form with product selection, quantity and reference fields

The Stock Out form removes inventory. It validates that the quantity is positive and does not exceed the current balance, and it offers a customer or department field whose value is prefixed to the transaction notes. This is a small design decision with a practical benefit: the audit trail records not only that stock left the store but also where it went, without requiring a separate issue document. Both forms show a busy state while the request is in flight, disable the submit button and display the server's error message if the request fails. On success, the user is taken to the transaction ledger and a toast confirms the operation.

The interface does not attempt to calculate the new balance locally. It sends the intent to the server and displays the result, because the balance may have changed since the page was loaded. This avoids a stale-browser problem in which two users issue the same stock based on the same displayed quantity.

5.5 Audit Trail, Reports and Low-Stock Queue

The Transactions page is the audit trail. Each row shows the timestamp, product name and code, movement type, quantity, the previous and new balance, the user who performed the movement and the reference. The arrow between the two balances makes the effect of the movement immediately readable. The search box matches product names, product codes, reference numbers and notes, so a delivery note or an issue reference can be traced directly. Figure 5.5 shows the ledger with the seeded movements.

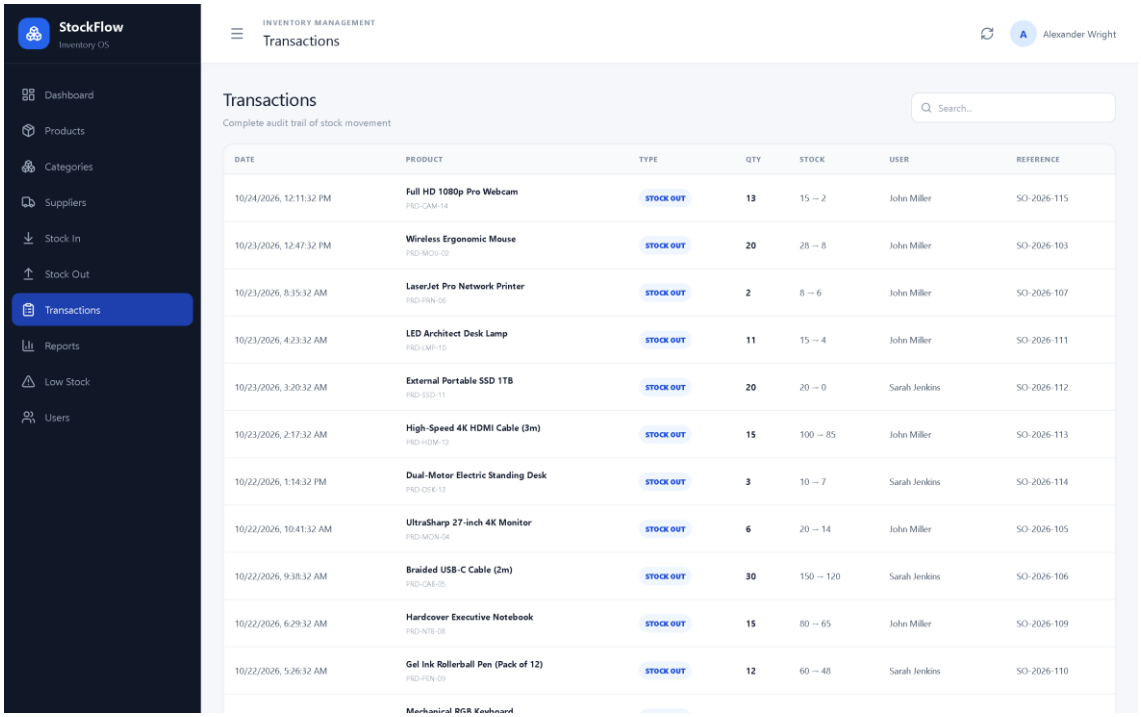


Fig 5.5: Transaction ledger with balances, users and references

The Reports page offers four tabs: Inventory, Stock movement, Low stock and Suppliers. The inventory report lists products with their category, supplier, quantity and prices; the stock movement report lists individual movements with their type, quantity and user; the low-stock report lists products at or below the minimum with a suggested reorder quantity; and the supplier report aggregates product counts, stock quantities and inventory value by supplier. Each tab is generated from the live database on selection, and the Refresh action re-runs the current report. Figure 5.6 shows the inventory report.

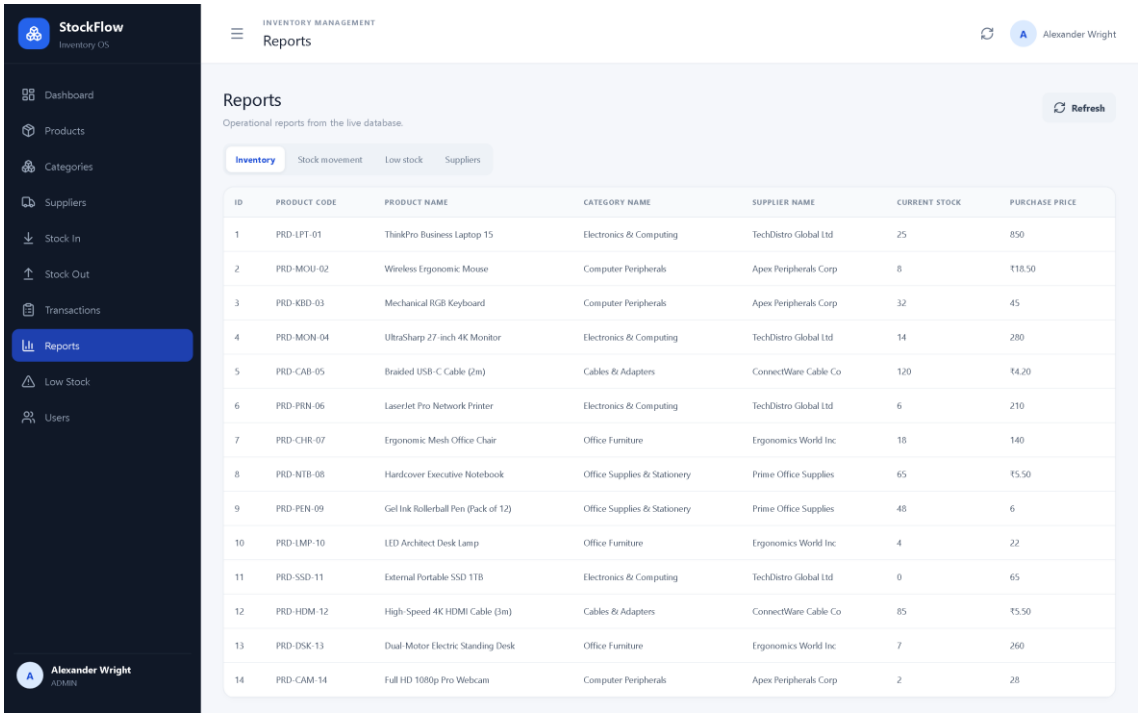


Fig 5.6: Reports page with inventory, movement, low-stock and supplier tabs

The Low Stock page is a focused replenishment queue. It lists the product, current quantity, minimum level, suggested reorder quantity, supplier and status, sorted by the most urgent item. The suggestion is calculated as maximum stock minus current stock, clamped at zero, so it refills the item towards its ceiling rather than merely to its minimum. Figure 5.7 shows the queue for the demonstration data.

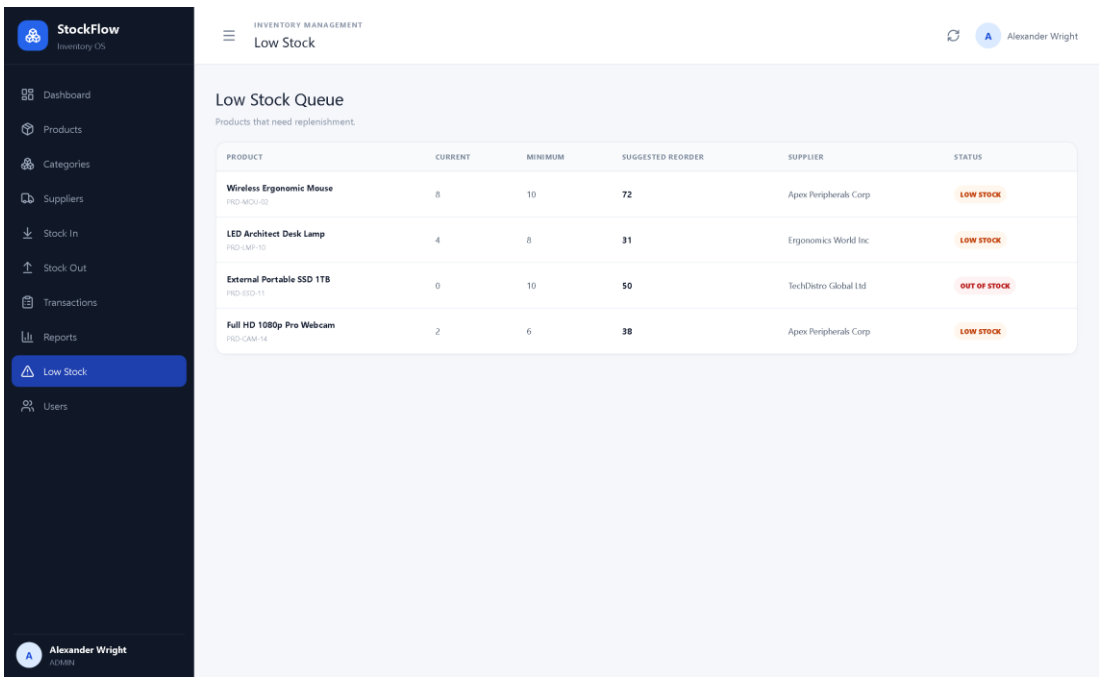


Fig 5.7: Low-stock queue with suggested reorder quantities

5.6 Users and Access Control

The Users page is visible only to administrators. It lists the name, e-mail, role and active status of each account and allows an administrator to create a user, change a role, reset a password, deactivate an account or delete it. Passwords are never stored in plain text: bcrypt produces a salted hash, and the login endpoint compares the submitted password with that hash. The API issues a signed JSON Web Token containing the user identifier and role; protected endpoints resolve the token, load the user, reject an inactive account and, for administrative routes, require the ADMIN role.

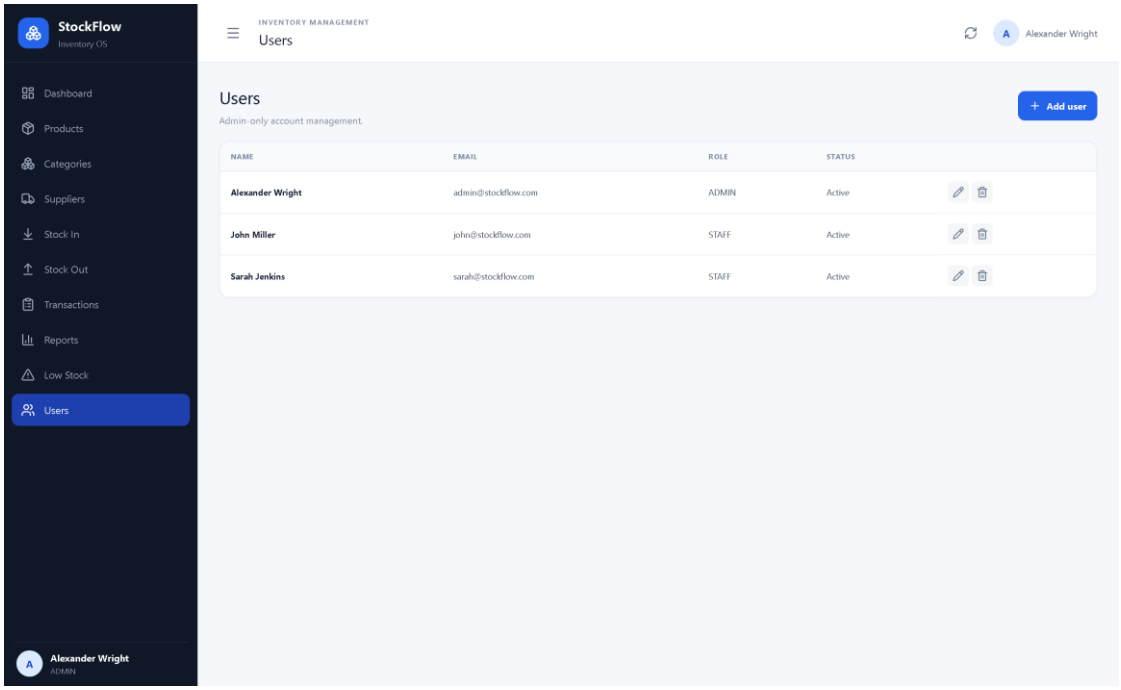


Fig 5.8: Administrator-only user management

Table 5.3: Roles and permissions

Capability	ADMIN	STAFF
View dashboard, catalogue, movements and reports	Yes	Yes
Record stock in and stock out	Yes	Yes
Create, edit and delete products	Yes	No
Create and edit categories and suppliers	Yes	No
Manage users and roles	Yes	No

Two self-protection rules prevent an administrator from locking the organisation out of its own administration: an administrator cannot demote or deactivate their own account, and cannot delete it. The implementation also keeps a demo-session fallback: when a request arrives without a bearer token, the API resolves the first active administrator, creating the demo account if the database is empty. This is what allows the delivered interface to open

without a login page. The token flow remains fully functional and is exercised by the automated tests, but a production deployment should disable the fallback and require sign-in.

5.7 Data Storage and Persistence

All business data is stored in backend/stockflow.db. The file is created automatically on startup if it does not exist, and SQLAlchemy creates the five tables from the model metadata. Every connection enables PRAGMA foreign_keys = ON, so the database enforces the relationships described in Chapter 4 rather than relying on application code alone. Each table carries created_at and updated_at timestamps; the transaction table carries created_at only, because a ledger entry is never edited.

The supplied seed script populates the database with three users, five categories, five suppliers, fourteen products and twenty-nine transactions spread over the previous week. The seed is idempotent: it checks for the demo administrator and exits if the data is already present, so it can be shipped with a project without duplicating records. The script is also the quickest way to restore a known demonstration state after experiments.

Because SQLite stores the database in a single file, backup is a file copy and the whole project can be moved between machines. The trade-off is that SQLite allows one writer at a time; this is acceptable for a small team and for the evaluation workload, but a larger deployment should move to PostgreSQL. The SQLAlchemy models and most of the service code would remain unchanged.

5.8 Responsive Interface

The layout is built with CSS grid and flexbox and is adjusted by two media queries. Below 1100 pixels the metric cards change from six columns to three, the dashboard panels stack and the category cards use two columns. Below 760 pixels the sidebar collapses to a 72-pixel icon rail, the card grid becomes two columns, the toolbar stacks vertically, the search field takes the full width and tables scroll horizontally inside their container. The same application therefore works on a laptop, a tablet and a phone without a separate mobile build. Figure 5.9 shows the dashboard at a phone width.

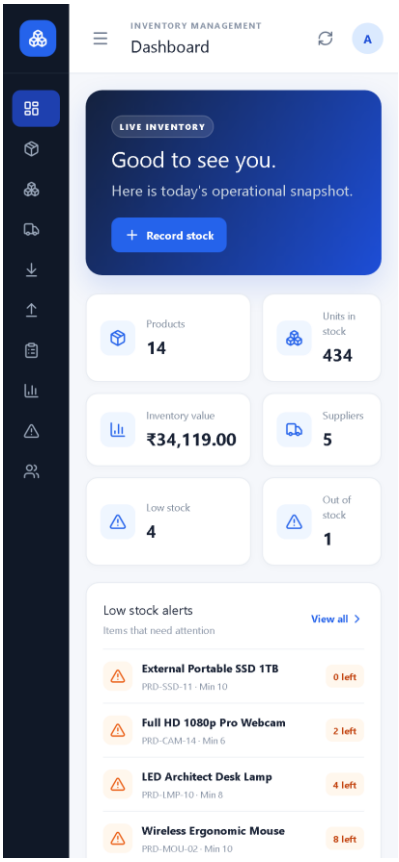


Fig 5.9: Dashboard at a phone-width viewport

5.9 How to Run

The application runs locally with two processes. The backend must be started first so that the dashboard can load its data:

- Backend: open a terminal in backend, install the requirements with pip install -r requirements.txt, then run python -m uvicorn app.main:app --reload --port 8000. The API is available at http://127.0.0.1:8000 and its interactive documentation at /docs.
- Frontend: open a second terminal in frontend, run npm install once, then npm run dev. The application opens at http://localhost:5173.
- Windows shortcut: start-backend.bat and start-frontend.bat perform the same steps from the project root.
- Demo data: the bundled database is already seeded. To rebuild it, stop the backend, delete backend/stockflow.db, then run python seed.py in the backend folder.
- Sign-in: the interface opens as the demo administrator. The login endpoint remains available at POST /api/auth/login for API clients that need a token.

The backend prints a startup message when the tables are ready, and the frontend prints the local URL. If the API is stopped, the interface shows a clear message with a Retry button instead of a blank page.

Chapter 6 Testing and Evaluation

6.1 Verification Approach

The system was evaluated at two levels. At the code level, an automated pytest suite exercises the REST API against an in-memory SQLite database, so each test starts from a known state and does not depend on the demonstration file. At the interface level, every screen was walked through feature by feature: records were created, edited, searched and deleted; stock was received and issued; invalid operations were attempted; and the results were compared with the expected behaviour. The frontend production build, the TypeScript compiler and the linter were also run as part of the evaluation.

The environment used for the evaluation was an ordinary development machine running Windows with Python 3.14.5 and Node.js 24.16.0, with the frontend served by Vite 8 and the API by Uvicorn on loopback ports 5173 and 8000 respectively. The browser was Google Chrome. The results describe this environment; a different machine or database size may produce different timings.

6.2 Functional Verification

Table 6.1 lists the functional checks that were performed through the interface and the API. Each check states what was done, the expected result and the observed outcome.

Table 6.1: Functional verification results

Function	What was checked	Expected behaviour	Result
Application startup	Frontend opened with the backend running and with it stopped	Dashboard loads; a clear retry message appears when the API is unavailable	Pass
Catalogue	Products page after seeding	All products, categories, suppliers and status badges are displayed	Pass
Create product	New product with category, supplier and opening stock	Product is saved, opening transaction is created and the row appears	Pass
Duplicate code	Create or edit with an existing product code	Request is rejected with a readable message	Pass
Edit product	Change the name and selling price of a product	Values are updated and the catalogue reloads in place	Pass
Delete product	Delete a product that has movements	Product and its dependent transactions are removed; the page stays on the catalogue	Pass
Stock In	Receive five units of a test product	Balance increases from 3 to 8 and a STOCK_IN row is appended	Pass
Stock Out	Issue five units of the same product	Balance decreases from 8 to 3 and a STOCK_OUT row is appended	Pass
Insufficient stock	Issue more units than the available balance	Request is rejected with the available quantity in the message	Pass

Function	What was checked	Expected behaviour	Result
Low-stock queue	Products at or below the minimum level	Queue lists the items with current, minimum and suggested reorder quantities	Pass
Dashboard	Cards after the test movements	Totals, inventory value and alert counts agree with the database	Pass
Search	Search products by name and transactions by reference	Only matching rows are returned	Pass
Reports	All four report tabs	Each tab loads live data and returns HTTP 200	Pass
Users	Create, edit and delete a user as administrator	Account is updated and the list refreshes without leaving the page	Pass
Persistence	Reload the browser after changes	Data is still present and the dashboard reflects the new balances	Pass

Result: all functional checks passed in the test environment.

6.3 Automated Backend Tests

The automated suite contains eight tests. It covers authentication, role enforcement, product validation, the complete stock-in and stock-out flow, insufficient-stock rejection, low-stock and reorder calculations, and dashboard statistics. The suite is run with pytest and uses an in-memory database with a test administrator and a test staff account.

Table 6.2: Automated test results

Test	Behaviour verified	Result
test_login_success	Valid credentials return an access token and the user profile	Pass
test_login_invalid_password	A wrong password returns 401 with a readable message	Pass
test_rbac_admin_only_users_endpoint	Administrators can list users; staff receive 403	Pass
test_product_crud_and_validation	Product creation, validation, update and admin-only deletion	Pass
test_stock_in_and_stock_out_flow	Both movement types update the balance and append transactions	Pass
test_insufficient_stock_rejection	An issue above the available balance is rejected	Pass
test_low_stock_and_reorder_calculation	Low-stock alerts and the suggested reorder quantity	Pass
test_dashboard_stats	Dashboard aggregates are returned with the expected fields	Pass

The final run reported 8 passed, 0 failed and 15 warnings; the warnings are deprecation notices from the test tooling and the use of timezone-naive timestamps, not failures. The frontend production build completed successfully (TypeScript and Vite), producing a 305 kB JavaScript bundle (96 kB gzip) and a 14.8 kB stylesheet (4.2 kB gzip). The linter reported zero errors and four advisory style warnings.

6.4 Performance Observations

Response times were measured on the loopback interface with a warm process, using fifteen consecutive requests per endpoint after the data had been seeded. Table 6.3 reports the

median and the 95th percentile. These figures describe the demonstration workload and are not a benchmark against a production database.

Table 6.3: Observed API response times (local machine, 15 requests per endpoint)

Endpoint	Median (ms)	95th percentile (ms)
GET /api/health	0.9	1.0
GET /api/dashboard/stats	2.2	2.4
GET /api/products (100 rows)	5.3	6.1
GET /api/inventory/transactions (100 rows)	6.7	7.3
GET /api/reports/inventory	4.6	5.1
GET /api/reports/stock-movement	6.1	6.6
GET /api/reports/low-stock	2.7	5.2
GET /api/users	2.2	2.8

All endpoints answered within single-digit milliseconds for this data set, and the browser rendered the resulting tables without a noticeable delay. The most expensive operations are the transaction and movement reports, because they scan every movement and resolve product and user names through relationships. The costs are acceptable at the demonstration scale; larger data sets would benefit from indexes on the date columns and from server-side pagination in the report endpoints.

6.5 Data Integrity Checks

Several checks specifically targeted the rules that protect the ledger. Deleting a category that still contains products returned HTTP 400 with a message explaining that the four products must be reassigned first; deleting a supplier in the same situation returned the equivalent message. Deleting a test product removed its two movements, confirming the cascade. An invalid login returned 401, and a staff token was refused access to the users endpoint with 403. Creating a product with a duplicate code and issuing more stock than was available both returned validation errors without changing the balance. After each check, the product counts and transaction totals were read directly from SQLite and matched the interface.

6.6 Threats to Validity and Limitations

The evaluation should be read with the following limitations in mind:

- Demonstration authentication: the interface opens as the demo administrator, so role restrictions were verified through the API and the test suite rather than by signing in

and out of the UI. A production deployment must require authentication and disable the demo fallback.

- Single-node database: SQLite serialises writes and the whole database is one file. Concurrent stock operations from many users were not stress-tested.
- Limited data volume: the seeded data set is small. Query plans and response times at tens of thousands of transactions were not measured.
- No frontend automation: the interface was verified manually. There is no browser test suite to catch regressions in the React components.
- Pagination in the interface: the API supports pagination, but the current screens request the first 100 records. A larger catalogue would need page controls.
- No notification channel: low-stock information is visible when the application is opened; there are no e-mail, SMS or push alerts.
- Single organisation: there is no tenant isolation, and the data model assumes one warehouse and one set of categories.
- No concurrency control: two simultaneous issues could both read the same balance before either writes. A production system should use row-level locking or an atomic conditional update.
- Functional rather than longitudinal evaluation: the study shows that the features work, not that they improve adherence to inventory policy over months of use.

Chapter 7 Conclusion and Future Work

StockFlow provides a complete, working solution for the inventory problem described in Chapter 3. It centralises products, categories, suppliers and movements in one application, keeps a trustworthy balance for every product, records every receipt and issue with the user and reference, derives stock status and reorder quantities automatically, and presents the result through a dashboard, an audit ledger and four reports. Administrative actions are separated from everyday stock work, and the interface adapts to desktop and mobile screens.

The objectives stated in Chapter 1 were met. The system records stock-in and stock-out movements atomically, rejects invalid quantities and duplicate codes, calculates the inventory value and low-stock counts from live data, produces reorder suggestions from the maximum level, and keeps the data available between sessions in SQLite. The automated suite and the manual walkthrough confirm the main flows, and the implementation remains small enough to be understood and extended by a student team.

The evaluation also exposed the boundaries of the current version. The demonstration opens without a login, the database is embedded and single-writer, the interface fetches at most one hundred rows per list, and there is no notification channel or concurrency control. These are deliberate scope choices for a local academic project, not hidden defects, and each has a clear path in the roadmap below.

Future Work

The current application covers the core inventory workflow. The enhancements in Table 7.1 would turn it into a more complete operational platform while preserving the same data model and interface style.

Table 7.1: Proposed enhancements

Enhancement	Description
Barcode and QR support	Scan a product code during receipt, issue and stock counting to reduce typing and mis-identification.
Purchase orders	Raise a purchase order to a supplier, receive it partially or fully, and reconcile the received quantity with the ordered quantity.
Multi-warehouse operation	Store balances per location, transfer stock between warehouses and report on each site separately.
Automated alerts	Send e-mail or chat notifications when a product crosses its minimum level or when a daily summary is due.
Cloud deployment	Move the API and PostgreSQL database to a hosted environment with HTTPS, backups and monitoring.

Enhancement	Description
Mobile and offline support	Package the interface as a progressive web application with offline caching for stock counts.
Demand forecasting	Use historical movements to estimate future demand and recommend order quantities and reorder points.
Supplier portal	Allow suppliers to confirm deliveries, upload invoices and view their own product and performance data.
Attachments and documents	Attach invoices, delivery notes and photographs to transactions for a complete documentary trail.
Granular permissions	Add roles such as storekeeper, purchase officer and auditor, each with a narrower set of allowed actions.
Audit export and retention	Export the ledger to CSV or PDF and define retention and archival rules for historical movements.
Frontend tests and CI	Add browser-level tests and a continuous integration pipeline that runs the backend suite, the type checker and the production build on every change.

With these enhancements, StockFlow can grow from a well-scoped academic project into a practical inventory platform for a small organisation, while keeping the clarity, validation and auditability that make the current version useful.

References

[1] FastAPI, “FastAPI Documentation”. https://fastapi.tiangolo.com/

[2] React Team, “React Documentation”. https://react.dev/

[3] Microsoft, “TypeScript Handbook”. https://www.typescriptlang.org/docs/handbook/

[4] Vite, “Vite Guide”. https://vite.dev/guide/

[5] SQLAlchemy, “SQLAlchemy 2.0 Documentation”. https://docs.sqlalchemy.org/en/20/

[6] SQLite, “SQLite Documentation”. https://www.sqlite.org/docs.html

[7] Pydantic, “Pydantic Documentation”. https://docs.pydantic.dev/

[8] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT), RFC 7519”, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519

[9] OWASP, “Password Storage Cheat Sheet”. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[10] Mozilla, “MDN Web Docs: HTTP”. https://developer.mozilla.org/en-US/docs/Web/HTTP

[11] Mozilla, “MDN Web Docs: Cross-Origin Resource Sharing (CORS)”. https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

[12] Tailwind Labs, “Tailwind CSS Documentation”. https://tailwindcss.com/docs

[13] Lucide, “Lucide Icons”. https://lucide.dev/

[14] pytest, “pytest Documentation”. https://docs.pytest.org/

[15] Axios, “Axios Documentation”. https://axios-http.com/docs/intro

[16] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2”, 2023. https://www.w3.org/TR/WCAG22/

Appendix A REST API Summary

The backend exposes the following endpoints under the /api prefix. Token means that the endpoint accepts a bearer token, or the demo session used by the delivered interface; Admin means that the ADMIN role is required. The interactive documentation at /docs lists the request and response schemas and is the authoritative reference; /openapi.json exposes the same schema for tooling.

Table A.1: REST endpoints

Area	Method and path	Purpose	Access
Authentication	POST /auth/login	Authenticate and return a token	Public
Authentication	GET /auth/me	Return the current user	Token
Users	GET /users	List accounts	Admin
Users	POST /users	Create an account	Admin
Users	PUT /users/{id}	Update name, role, status or password	Admin
Users	DELETE /users/{id}	Delete an account	Admin
Categories	GET /categories	List categories with product counts	Token
Categories	POST /categories	Create a category	Admin
Categories	PUT /categories/{id}	Update a category	Admin
Categories	DELETE /categories/{id}	Delete an unused category	Admin
Suppliers	GET /suppliers	List suppliers with product counts	Token
Suppliers	POST /suppliers	Create a supplier	Admin
Suppliers	PUT /suppliers/{id}	Update a supplier	Admin
Suppliers	DELETE /suppliers/{id}	Delete an unused supplier	Admin
Products	GET /products	Search, filter, sort and paginate	Token
Products	GET /products/{id}	Return one product	Token
Products	POST /products	Create a product with optional opening stock	Admin
Products	PUT /products/{id}	Update a product	Admin
Products	DELETE /products/{id}	Delete a product and its movements	Admin
Inventory	POST /inventory/stock-in	Receive stock and append a transaction	Token
Inventory	POST /inventory/stock-out	Issue stock and append a transaction	Token
Inventory	GET /inventory/transactions	List and search the ledger	Token
Dashboard	GET /dashboard/stats	Return the six aggregate values	Token
Dashboard	GET /dashboard/low-stock-alerts	Return products below the minimum	Token
Dashboard	GET /dashboard/reorder-recommendations	Return suggested order quantities	Token
Reports	GET /reports/inventory	Inventory valuation report	Token
Reports	GET /reports/stock-movement	Movement report with optional filters	Token
Reports	GET /reports/low-stock	Low-stock report with reorder suggestions	Token
Reports	GET /reports/suppliers	Supplier summary report	Token
Health	GET /health	Service health check	Public